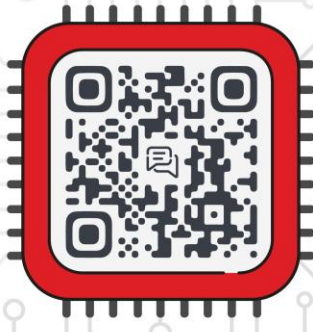




نظري

1

25

3000 sp

كلية الهندسة المعلوماتية

السنة الثالثة

Control Unit Design

(1)

د. سيرا أستور



RB Informatics; 13/10/2025

بنیان الحواسيب 2

بسم الله نبدأ وعلى الله نتوكل في مقررنا مقرر بنیان الحواسيب 2.

نبدأ رحلة علمية نرجو أن تكون ممتعة ومثيرة، سنبدل جهدنا لتكون المحاضرات عوناً لكم فلا تنسونا من صالح دعائكم. ♥

في هذه المحاضرات سنبدأ من الصفر وسنصل إلى الدارة، أي سنبدأ من المتطلبات ومن ثم نصل إلى برنامج كامل يعطينا output.

فلنبدأ رحلتنا بتصميم ال Control Unit Design كما أننا سوف نتذكر بعض التفاصيل من مادة بنیان الحواسيب 1 دعونا نبدأ !

1. مكونات المعالج الأساسية : المعالج يتألف من Data path و Control

■ Data path "مسار أو طريق المعطيات"

مجموعة مكونات نسميها عادة sequential.

تتكون من عناصر مثل: الذاكرة، المسجلات ، و (adders)، وحدة الحساب والمنطق (Alu)، و أسلاك التوصيل (communication buses).

كل خطوة من خطوات تنفيذ التعليم (جلب، فك تشفير، تنفيذ، حفظ النتيجة) تحتاج إلى نقل بيانات بين هذه العناصر (data transfer) لذلك يجب أن تكون هناك مسارات واضحة تربط بين الذاكرة والمسجلات وال Alu.

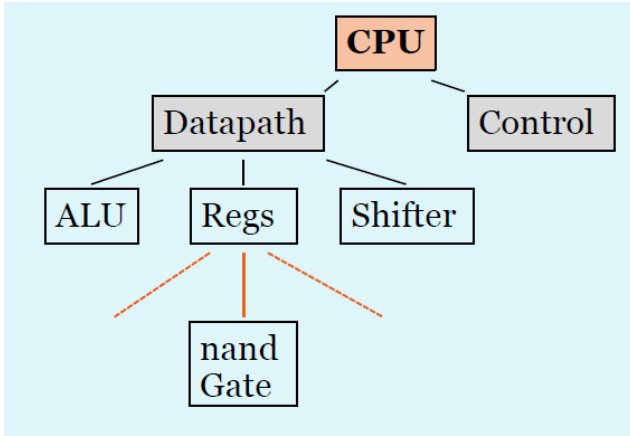
■ التحكم (Control): مهمة وحدة التحكم هي تنظيم عمل الـ Data path في كل خطوة، تقوم بإرسال إشارات

تحكم تحدد اتجاه تدفق البيانات على الـ communication buses و تختار الوظائف المطلوبة من Alu و الذاكرة. يتم توليد هذه الإشارات من وحدة تحكم تتكون من one or more Finite-state Machines

⚡ ملاحظة:

كل تعليمة لها مسار منفصل لأن كل تعليمة تقوم بعمل مختلف. ولكن إذا كان لدي عدد كبير من التعليمات، يصبح لدينا شبكة طرق والذي يرشد التعليمة إلى أي طريق هو Control

عملية التصميم (Design process)



التصميم هو عملية إبداعية وليست مجرد طريقة بسيطة.

يُفهم التصميم من خلال مكوناته وكيف تم تجميعها مثلاً
لتشكيل النظام

تذكرة أربسية :

ALU : وحدة الحساب والمنطق تنفذ العمليات الرياضية والمنطقية.

CPU : وحدة المعالجة المركزية، تدير تنفيذ العمليات.

Data Path : مسار البيانات بين المكونات.

Regs : المسجلات التي تخزن البيانات مؤقتاً.

shifter : وحدة الإزاحة، تغير مواقع البتات.

NAND Gate : بوابة منطقية تُستخدم في بناء الدوائر.

control : وحدة التحكم تنظم عمل كل المكونات .

يوجد طريقتان للتصميم:

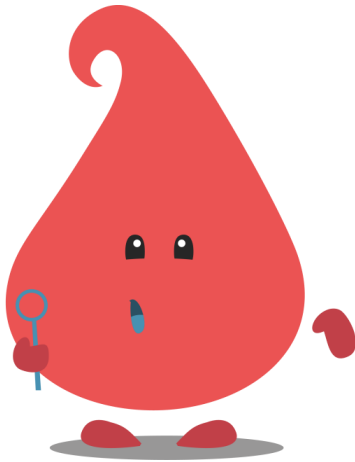
1. من الأعلى إلى الأسفل (Top - Down):
نبدأ بوظائف معقدة ونفكها إلى وظائف أبسط (أي نمسك العنصر الكبير و نجزأه إلى عناصر أصغر).
2. من الأسفل إلى الأعلى (Bottom-up):
نبدأ ببناء وحدات بسيطة ثم نركبها معاً لتشكيل نظام أكثر تعقيداً. (نبدأ بتصميم العنصر الصغير).



كل الأنظمة الحالية هي أنظمة معقدة لا نصممها دفعة واحدة وإنما نقسمها إلى أقسام.

وهنا السؤال كيف يمكنني أن أحافظ على الحالة السابقة أحتاج الى تخزين

ماهي أهم الدارات البسيطة على مستوى الدارات المنطقية التي تقوم بالتخزين،
هي flip-flop و العدادات



Defining control (تعريف التحكم)

Sequential Systems (الأنظمة التسلسلية) ⬅

هي أنظمة تحتوي على حالة داخلية يتم تخزينها في عناصر ذاكرة داخلية. وتعرف هذه العناصر بـ state elements

■ سلوك النظام يعتمد على أمرين :

- 1 - مجموعة مدخلات التي يتم تزيدها النظام بها .
- 2 - محتوى الذاكرة الداخلية أو حالة النظام الحالية .

⚠ ملاحظة مهمة :

لا يمكن وصف النظام التسلسلي باستخدام جدول الحقيقة فقط. لأنه لا يعتمد فقط على المدخلات، بل أيضاً على الحالة السابقة. لذلك، يتم وصف الأنظمة التسلسلية باستخدام الـ FSM (Finite-State Machine). وهو نموذج يحدد كيف تتغير الحالة بناءً على المدخلات

FSM (Finite-State Machine):

يتكون من مجموعة من الحالات والتعليمات التي توضح كيف تنتقل من حالة إلى أخرى. سنستخدم جزأين من التوابيع:

1. Next-state function ("دالة الحالة التالية"):

هي دالة منطقية مركبة تأخذ المدخلات والحالة الحالية وتحدد الحالة التالية للنظام.
أي أنا موجود في حالة ما هي الحالة الثانية التي سوف أنتقل إليها، إلى أين سأصل، مثل:
 $PC + 4$ نأخذ الـ PC ونجمع له 4 ونضعه بالـ PC.

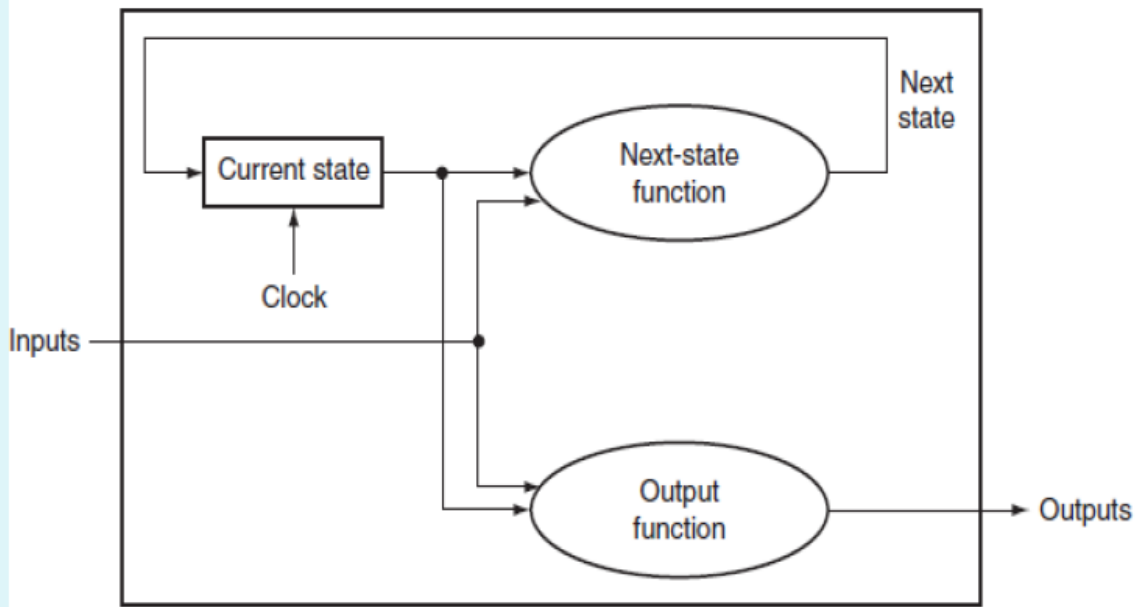
2. Output Function ("دالة المخرجات"):

تنتج مجموعة من المخرجات بناءً على الحالة الحالية والمدخلات.

إذا كان لدينا n بت مخصصة للتخزين، فهذا يعني أن عدد الحالات الممكنة هو 2^n حالة.

Finite-State Machine (FSM)

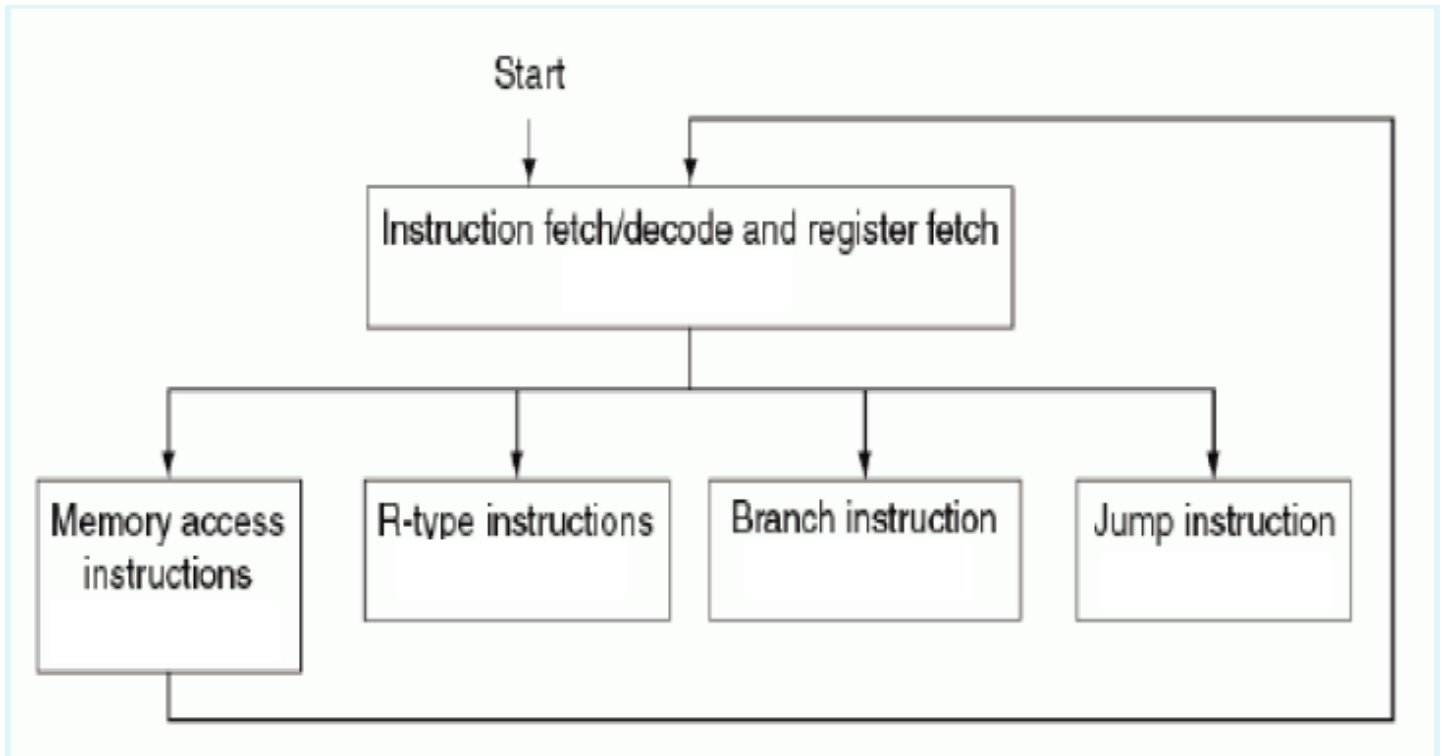
6

شرح الرسم:

نلاحظ أن ال inputs يتعلق باثنين ممكن أن يوصلني على output function و Next-state و هو السهم العائد و هو الذي يتعلق بال current state .



High-level view of FSM control for MIPS processor



هذا المخطط يوضح كيف يتم استخدام الـ FSM في معالج MIPS. حيث تقوم الـ FSM بتوليد إشارات التحكم المناسبة في كل حالة لتوجيه أجزاء المعالج مثل الذاكرة، المسجلات، و الـ ALU.

Design Implementations

تصميم وحدة التحكم باستخدام – FSM طرق تنفيذ دالة الحالة التالية:

عندما لا تتبع الحالة التالية التسلسل الطبيعي، يتم استخدام منطق إضافي لتحديد الحالة المطلوبة.

هناك طريقتان رئيسيتان لحساب الحالة التالية في: FSM

1. الطريقة الأولى: باستخدام منطق مخصص (ROM / PLA):

- يتم تمثيل دالة الحالة التالية باستخدام جدول منطقي داخل : ROM (Read-Only Memory) يحتوي على كل الحالات والانتقالات.
- PLA (Programmable Logic Array) : مصفوفة قابلة للبرمجة تنفذ المنطق المطلوب.
- هذه الطريقة تسمح بسرعة كبيرة في تصميم الانتقالات بين الحالات.
- تستخدم عندما تكون الحالات غير متسلسلة أو تعتمد على مدخلات متعددة.
- غير مرنة لأنه يتم تصميم الدارة فيزيائياً (نحتاج تعديل البوابات)

2. الطريقة الثانية: باستخدام عداد (Counter):

- يتم استخدام عداد يقوم بزيادة الحالة الحالية تلقائيًا
- لكن إذا كانت الحالة التالية لا تتبع تسلسل يتم إدخال منطق إضافي لتحديد الحالة المطلوبة (مثل المقارنات أو شروط خاصة).

تذكرة من الدارات المنطقية

- الفرق الجوهرى بين الدارات المنطقية التتابعية (Sequential) والتوافقية (Combinational) هو أن التتابعية تعتمد على الزمن أو الحالة السابقة، بينما التوافقية تعتمد فقط على المدخلات الحالية.

لنقارن سوياً بين النوعين:

الدارات المنطقية التوافقية (Combinational Logic Circuits)

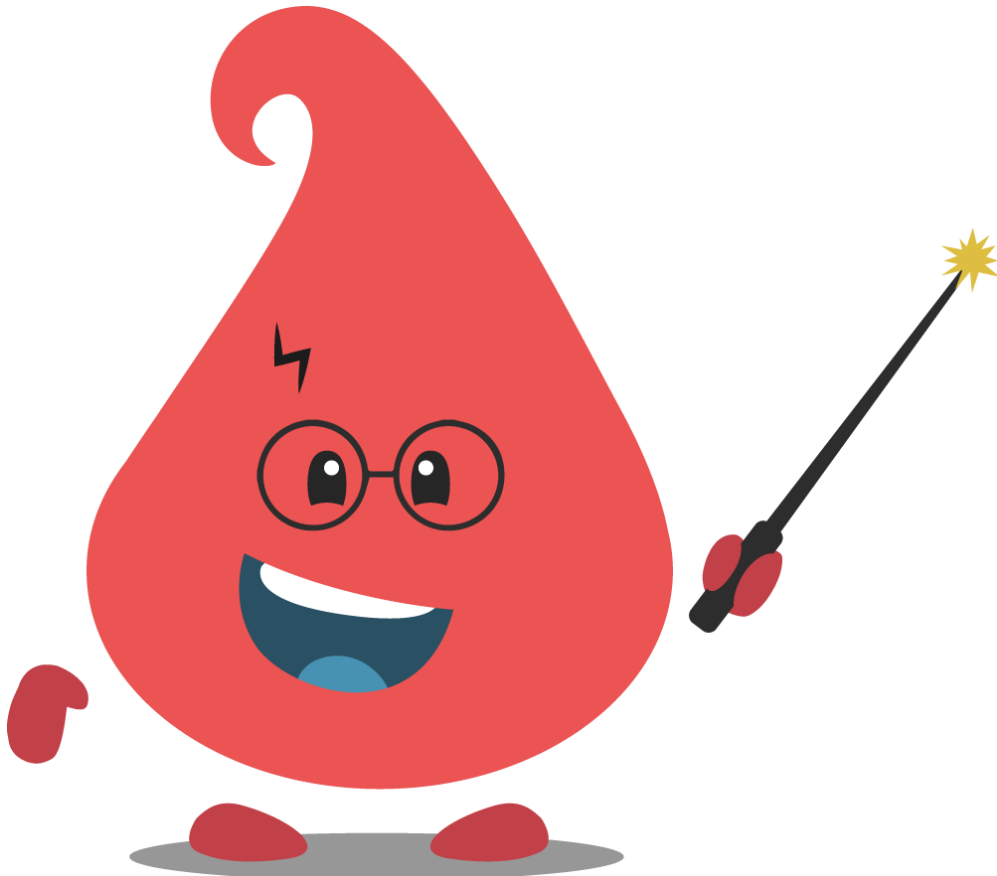
- الخرج يعتمد فقط على المدخلات الحالية، دون أي اعتبار للحالة السابقة.
- لا تحتوي على ذاكرة، وبالتالي لا يمكنها تذكر أي معلومات.
- أمثلة شائعة: الجامع (Adder)، المقارن (Comparator)، المفكك (Decoder)، المبدل (Multiplexer)
- الاستخدامات: العمليات الحسابية والمنطقية البسيطة التي لا تحتاج إلى تتبع الحالة.

الدارات المنطقية التتابعية (Sequential Logic Circuits)

- الخرج يعتمد على المدخلات الحالية والحالة السابقة (أي أن لها ذاكرة).
- تحتوي على عناصر تخزين مثل Flip-Flops أو Latches.
- تُستخدم لتصميم الآلات ذات الحالات (Finite State Machines).
- أمثلة شائعة: العدادات (Counters)، السجلات (Registers)، الذاكرة (RAM)، وحدات التحكم (Control Units)
- الاستخدامات: الأنظمة التي تحتاج إلى تتبع تسلسل الأحداث أو الحالات مثل المعالجات والأنظمة المضمنة

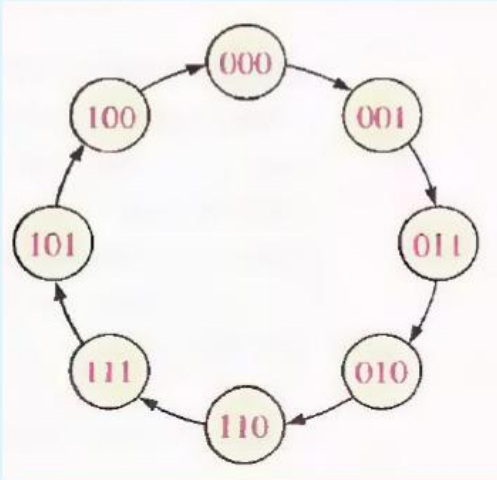
خطوات تصميم الـ FSM باستخدام العداد

1. ترميز الحالات (Encoding states):
إذا عندنا 10 حالات نحتاج 4 بت لتمثيلها.
مثال: الحالة 0 = 0000، والحالة 1 = 0001 وهكذا....
2. كتابة جدول الحقيقة (Truth table):
نكتب جدول يوضح العلاقة بين الحالة الحالية والمدخلات وما هي الحالة التالية وإشارة التحكم الناتجة.
3. توليد الدائرة المنطقية (logic circuit):
نستخدم جدول الحقيقة لتصميم دائرة منطقية تتحكم بالانتقال بين الحالات.
4. توصيل الـ (Flip Flops):
نربط 4 Flip Flops بين مخرجات الحالة "التالية" ومدخلات الحالة "الحالية" لتخزين الحالة وتحديثها في كل دورة ساعة



First Step & second Step

• First Step: State Diagram



• 2 Step: Next State Table

PRESENT STATE			NEXT STATE		
Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0

• مثال على ترميز الحالات باستخدام عداد Gray

السهم يمثل اتجاه الـ next state.

وكما نعلم أن الـ gray يتميز بعده بتغيير بت واحد فقط لكل نقطة فيوجد لدينا ثمان حالات

Step3 : Flip-Flop Transition Table

OUTPUT TRANSITIONS		FLIP-FLOP INPUTS	
Q_N	Q_{N+1}	J	K
0	→ 0	0	X
0	→ 1	1	X
1	→ 0	X	1
1	→ 1	X	0

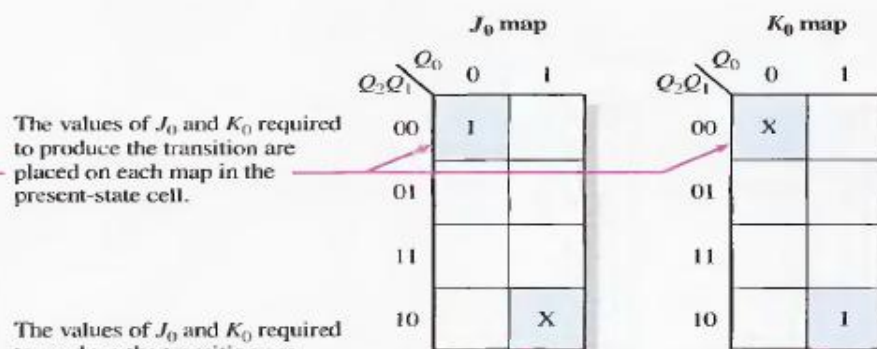
Q_N : present state
 Q_{N+1} : next state
X: "don't care"

كنا سابقاً في الدارات المنطقية نأخذ الـ input ونخرج منه الـ output. أما هنا فنحن نقوم بالعكس، نأخذ الـ output ونخرج منه الـ input. لذلك سنحتاج إلى Transition Table.

مثال للشرح : الحالة الثانية نقرأها إذا كان لدي صفر وفي الحالة التالية يصبح واحد،

ماذا يجب أن يكون الـ J أو الـ K

Step4 : Karnaugh Tables



Output Transitions		Flip-Flop Inputs	
Q_N	Q_{N+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

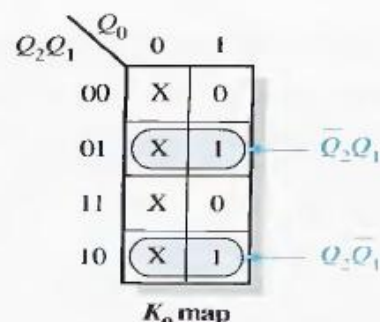
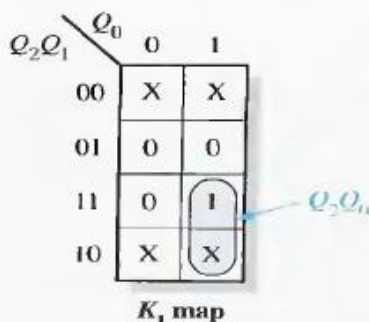
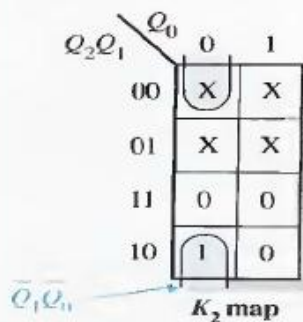
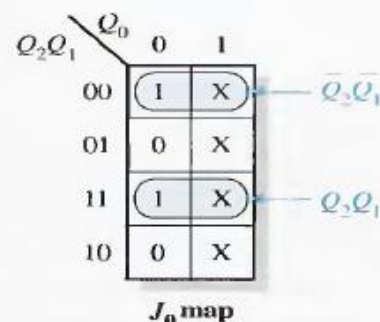
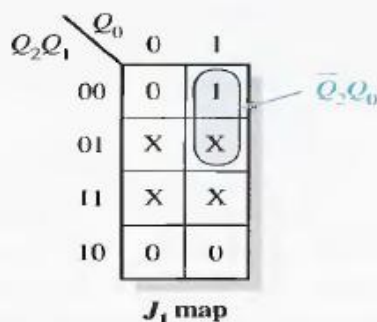
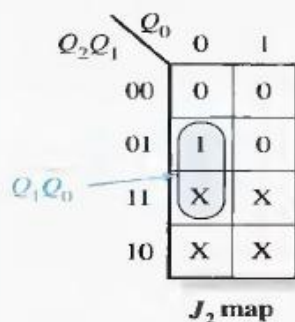
Flip-flop transition table

Present State			Next State		
Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0

Next-state table

For the present state 000, Q_0 makes a transition from 0 to 1 to the next state.

For the present state 101, Q_0 makes a transition from 1 to 0 to the next state.



Step5 : Logical Expressions for Flip-Flop inputs

$$J_0 = Q_2 Q_1 + \overline{Q_2} \overline{Q_1} = \overline{Q_2} \oplus \overline{Q_1}$$

$$K_0 = Q_2 \overline{Q_1} + \overline{Q_2} Q_1 = Q_2 \oplus Q_1$$

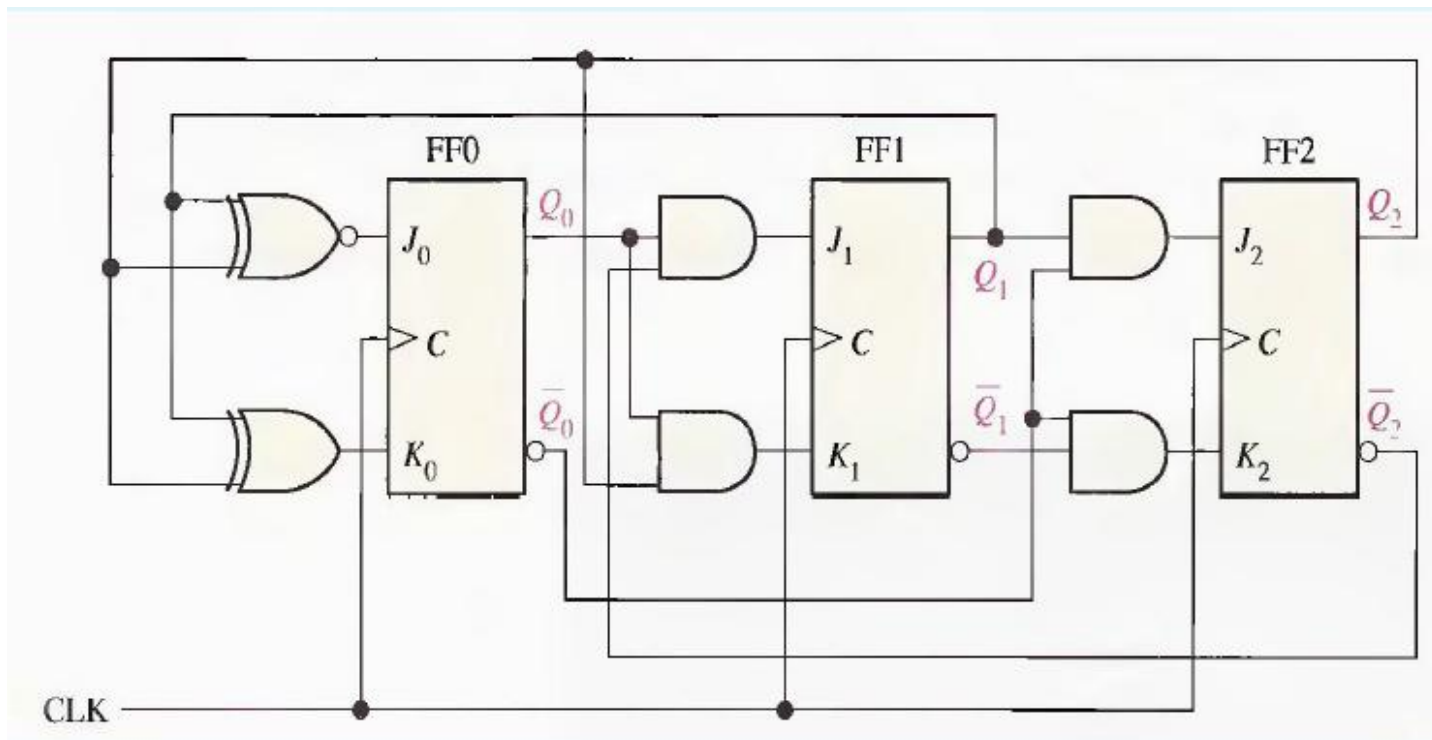
$$J_1 = \overline{Q_2} Q_0$$

$$K_1 = Q_2 Q_0$$

$$J_2 = Q_1 \overline{Q_0}$$

$$K_2 = \overline{Q_1} \overline{Q_0}$$

Step 6: Counter Implementation



نلاحظ في هذه الدارة أنه كل ما يأتي clk تنتقل الى Next State حسب ال Gray

تذكرة:

أنواع القلابات : D _ RX _ JK

FlipFlop Transition Tables

Inputs			outputs		
J	k	C	Q	$\overline{Q}$	
0	0	1	Q_0	$\overline{Q_0}$	No Change
0	1	1	0	1	Reset
1	0	1	1	0	Set
1	1	1	$\overline{Q_0}$	Q_0	Toggle

outputs		Inputs	
Q_N	Q_{N+1}	J	K
0	----	0	x
0	----	1	x
1	----	x	1
1	----	x	0

outputs		Inputs
Q_N	Q_{N+1}	D
0	----	0
0	----	1
1	----	0
1	----	1

J-K Truth Table

J-K Transition Table

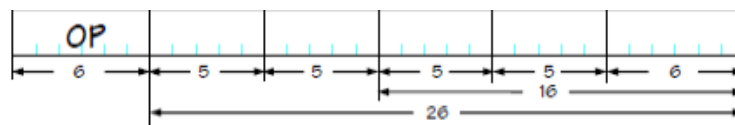
D Transition Table

The MIPS ISA

The MIPS instruction set as seen from a Hardware Perspective

Instruction classes distinguished by types:

- 1) 3-operand ALU
- 2) ALU w/immediate
- 3) Loads/Stores
- 4) Branches
- 5) Jumps



000000	r_s	r_t	r_d	shamt	func
--------	-------	-------	-------	-------	------

R-type: ALU with Register operands

$$\text{Reg}[rd] \leftarrow \text{Reg}[rs] \text{ op } \text{Reg}[rt]$$

001XXX	r_s	r_t	immediate		
--------	-------	-------	-----------	--	--

I-type: ALU with constant operand

$$\text{Reg}[rt] \leftarrow \text{Reg}[rs] \text{ op } \text{SEXT}(\text{immediate})$$

10X011	r_s	r_t	immediate		
--------	-------	-------	-----------	--	--

I-type: Load and Store

$$\text{Reg}[rt] \leftarrow \text{Mem}[\text{Reg}[rs] + \text{SEXT}(\text{immediate})]$$

$$\text{Mem}[\text{Reg}[rs] + \text{SEXT}(\text{immediate})] \leftarrow \text{Reg}[rt]$$

10X011	r_s	r_t	immediate		
--------	-------	-------	-----------	--	--

I-type: Branch Instructions

$$\text{if } (\text{Reg}[rs] == \text{Reg}[rt]) \text{ PC} \leftarrow \text{PC} + 4 + 4 * \text{SEXT}(\text{immediate})$$

$$\text{if } (\text{Reg}[rs] != \text{Reg}[rt]) \text{ PC} \leftarrow \text{PC} + 4 + 4 * \text{SEXT}(\text{immediate})$$

00001X	20-bit constant				
--------	-----------------	--	--	--	--

J-type: jump

$$\text{PC} \leftarrow (\text{PC} \& 0xf0000000) | 4 * (\text{immediate})$$

مجموعة تعليمات MIPS من منظور الهاردوير .

↪ أنواع التعليمات (Instruction classes)

1. :operand Alu-3

تعليمات تستخدم ثلاث معاملات وتنفذ عمليات حسابية أو منطقية باستخدام Alu

مثال : `add $t0, $t1, $t2`

2. :Alu with Immediate

التعليمات التي تستخدم معامل مباشر (قيمة ثابتة) مع وحدة الحساب والمنطق. (Alu)

مثال : `addi $t0, $t1, 5`

3. :Loads/stores

تعليمات مخصصة لتحميل البيانات من الذاكرة إلى المسجلات أو لتخزينها من المسجلات إلى الذاكرة.

مثال : `lw $t0, 0($t1)`

`sw $t0, 0($t1)`

4. :Branches

تعليمات تحقق شرطاً معيناً وتنتقل إلى عنوان محدد إذا تحقق هذا الشرط

مثال : `beq $t0, $t1, Label`

5. : Jumps

التعليمات التي تنفذ قفزة مباشرة إلى عنوان معين

مثال : `J Label`



Designing the Main Control Unit "Single Cycle"

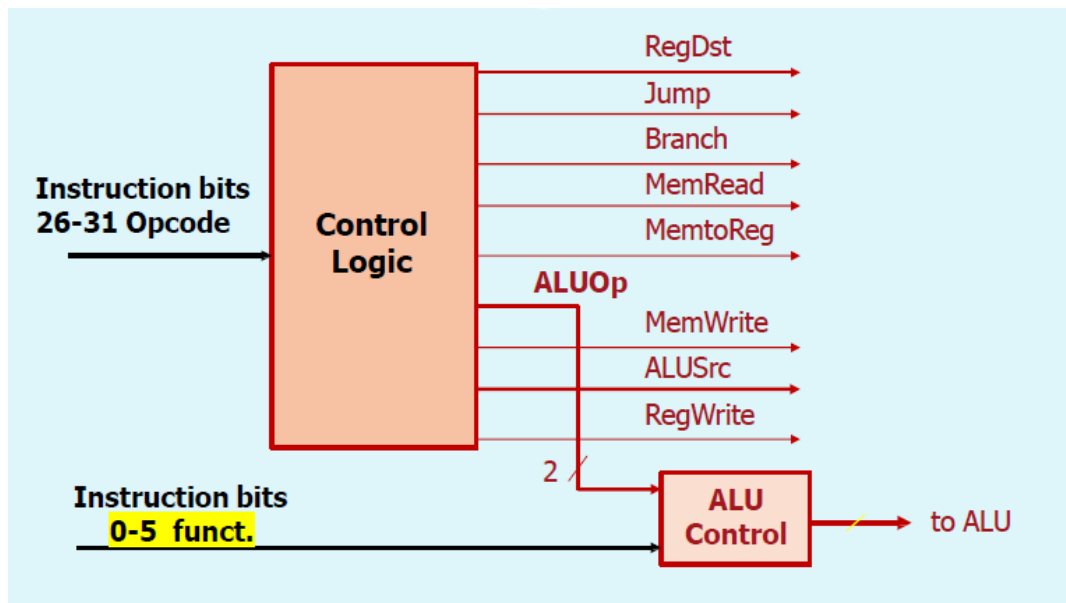
تصميم وحدة التحكم الرئيسية (Main Control Unit Design) :

3. التصميم متعدد المستويات (Multilevel Design) :

يكون لدينا وحدة تحكم خاصة بوحدة الحساب والمنطق (ALU) بالإضافة إلى وحدة التحكم الرئيسية. وحدة التحكم الرئيسية تقوم بتوليد ALUOp bits التي تستخدم كمدخلات لوحدة تحكم ALU و التي بدورها تولد الإشارات الفعلية للتحكم في ال ALU. يُعتبر هذا الأسلوب تقنية شائعة لأنه يُبسّط التصميم،

4. فائدته: في أن استخدام مستويات متعددة من التحكم يساعد على تقليل حجم وحدة التحكم الرئيسية، مما يجعل التصميم أبسط وأكثر كفاءة.

تصميم وحدة التحكم الرئيسية في دورة واحدة (Single Cycle)



يتم استخدام بتات التعليمات لتحديد نوع العملية المطلوبة

-البتات من 26 إلى 31 تمثل Opcode (رمز العملية).

-البتات من 0 إلى 5 (funct field) تستخدم في تعليمات R-type لتحديد العملية الدقيقة داخل ALU Control Unit
وحدة التحكم الرئيسية في تصميم "دورة واحدة" مسؤولة عن توليد إشارات التحكم مباشرة لكل تعليمة، حيث تولد إشارات مثل: **RegDst**, **Jump**, **Branch**, **MemRead**, **MemtoReg**, **ALUOp**, **MemWrite**, **ALUSrc**, و **RegWrite**.

تأخذ ALU control Unit مدخل ALUOp (2 بت) وحقل funct وتحدد العملية الفعلية التي يجب أن تنفذها ال ALU.

مثال على تصميم وحدة تحكم في ال ALU

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

نفترض أن وحدة ALU في MIPS تدعم 6 وظائف مختلفة تستخدم في أنواع التعليمات التالية :

- تعليمات النوع R-type.
- وتعليمات التحميل والتخزين (load/store) .
- وتعليمات الفروع مثل beq

و بما أن لدينا 3 حالات رئيسية، نحتاج إلى 2 بت لتشفيرها.

■ مثال :

يمكننا توليد مدخل تحكم مكون من 4 بت لوحدة ALU باستخدام وحدة تحكم صغيرة تُسمى ALU Control Unit مدخلات هذه الوحدة هي:

-حقل الوظيفة function field (funct) من التعليمات (البتات 5-0)
-وحقل التحكم ALUOp (2 بت).

دلالة ALUOp كالتالي:

- 00 :تمثل عملية الجمع (add) لتعليمات التحميل والتخزين.
- 01 : تمثل عملية الطرح (subtract) لتعليمات التفرع (beq)
- 10 : تمثل عملية يتم تحديدها من حقل funct لتعليمات النوع R.
- 11 غير مستخدمة (NOT used).

أما مخرجاتها فهي إشارة تحكم مكونة من 4 بت تتحكم مباشرة بوحدة ALU عبر توليد أحد التركيبات الثنائية الممكنة

	Instruction opcode	ALUOp	Instruction operation	Func field	Desired ALU action	ALU control input
Type 1	LW	00	load word	XXXXXX	add	0010
Type 2	SW	00	store word	XXXXXX	add	0010
Type 2	Branch equal	01	branch equal	XXXXXX	subtract	0110
Type 3	R-type	10	add	<u>100000</u>	add	0010
Type 3	R-type	10	subtract	<u>100010</u>	subtract	0110
Type 3	R-type	10	AND	<u>100100</u>	AND	0000
Type 3	R-type	10	OR	<u>100101</u>	OR	0001
Type 3	R-type	10	set on less than	<u>101010</u>	set on less than	0111

ALUOp		Func field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

ملاحظات مهمة :

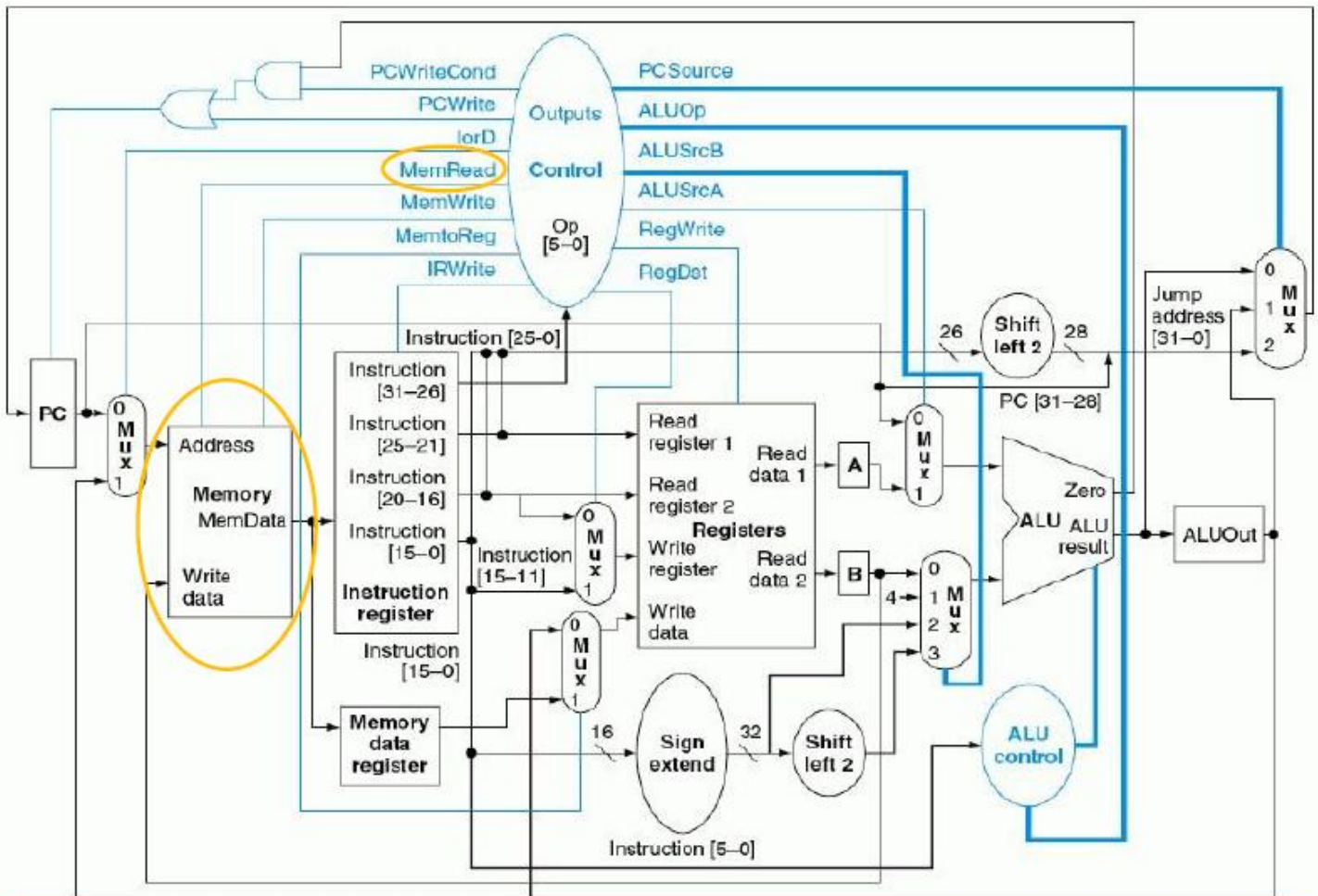
1. الحالة 11 من ALUOp غير مستخدمة، لذلك يمكن نستخدم أنماط مثل:
 x1 (أي أن البت الأول غير مهم، والثاني = 1)
 1x (أي أن البت الثاني غير مهم، والأول = 1) بدلاً من استخدام 01 و 10 بشكل مباشر في جدول الحقيقة.

2. فيما يخص حقل funct :

عندما يتم استخدام حقل funct لتحديد العملية في تعليمات النوع R، فإن أول بتين من هذا الحقل F5 و F4 دائماً تكون قيمتهما 10

لذلك، يُعتبر هذان البتان don't-care terms (غير مهمين)، ويتم استبدالهما بـ XX في جدول الحقيقة. والفائدة من ذلك هي تقليل التعقيد، حيث نركز فقط على البتات التي تؤثر فعلياً في تحديد العملية.

- حقل Op - رمز التشغيل موجود دائماً في البتات 31:26.
- السجلان المطلوب قراءتهما يتم تحديدهما دائماً في حقلي rs و rt في المواقع 25:21 و 20:16.
- هذا صحيح بالنسبة لتعليمات النوع R ، وتعليمات branch equal ، وتعليمات التخزين.
- السجل الأساسي لتعليمات التحميل والتخزين موجود دائماً في المواقع 25:21 (rs).
- الإزاحة ذات 16 بت لتعليمات الفرع المتساوي، التحميل والتخزين موجودة دائماً في المواقع 15:0.
- سجل الوجهة موجود في مكانين:
- في تعليمات التحميل: المواقع 20:16 (rt)
- في تعليمات النوع R: المواقع 15:11 (rd)
- نحتاج إلى MUX لاختيار أي حقل من التعليمات سيتم استخدامه.



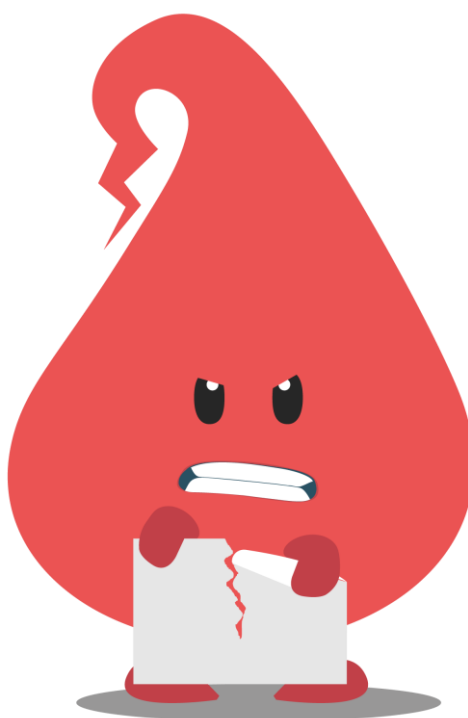
Multi-Cycle Control Lines

27

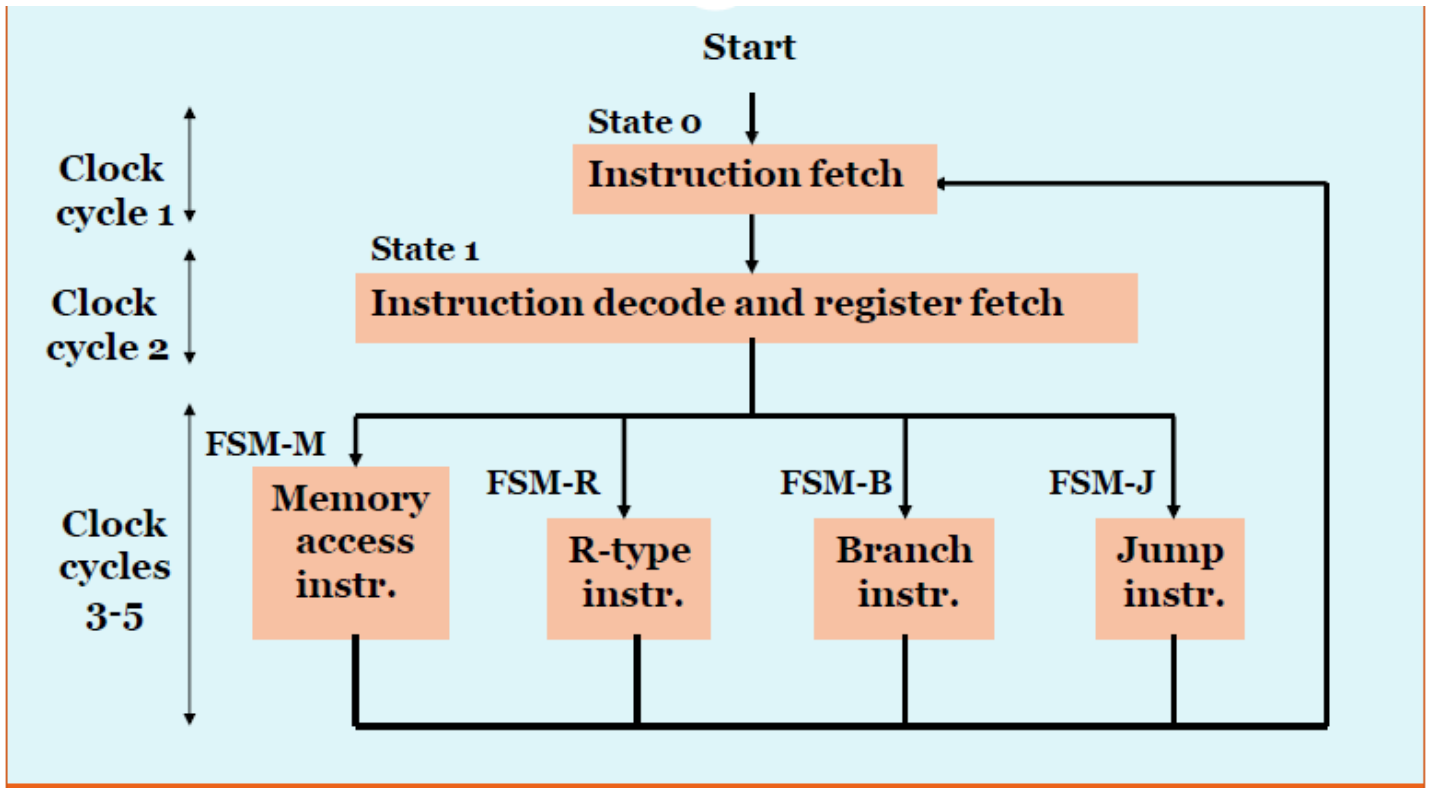
16 Control Lines

3 to 5 cycles for an Instruction

Step	R-type (4 cycles)	Mem. Ref. (4 or 5 cycles)	Branch type (3 cycles)	J-type (3 cycles)
Instruction fetch	$IR \leftarrow \text{Memory}[PC]; PC \leftarrow PC+4$			
Instr. decode/ Reg. fetch	$A \leftarrow \text{Reg}(IR[21-25]); B \leftarrow \text{Reg}(IR[16-20])$ $ALUOut \leftarrow PC + (\text{sign extend } IR[0-15]) \ll 2$			
Execution, addr. Comp., branch & jump completion	$ALUOut \leftarrow A \text{ op } B$	$ALUOut \leftarrow A + \text{sign extend } (IR[0-15])$	If (A==B) then $PC \leftarrow ALUOut$	$PC \leftarrow PC[28-31]$ $(IR[0-25] \ll 2)$
Mem. Access or R-type completion	$\text{Reg}(IR[11-15]) \leftarrow ALUOut$	$MDR \leftarrow M[ALUOut] \text{ or } M[ALUOut] \leftarrow B$		
Memory Read completion		$\text{Reg}(IR[16-20]) \leftarrow MDR$		



Control: Finite State Machine



تسلسل الحالات في تنفيذ التعليمات :

الحالة	الدورة الزمنية	الوظيفة المنفذة في هذه الحالة
0	الدورة 1	جلب التعليمات من الذاكرة إلى سجل التعليمات (IR)
1	الدورة 2	فك التعليمات وجلب السجلات المطلوبة A و B

نفذ التعليمات حسب نوعها:

FSM-M: تعليمات التحميل والتخزين (Load/Store)

FSM-R: تعليمات النوع R

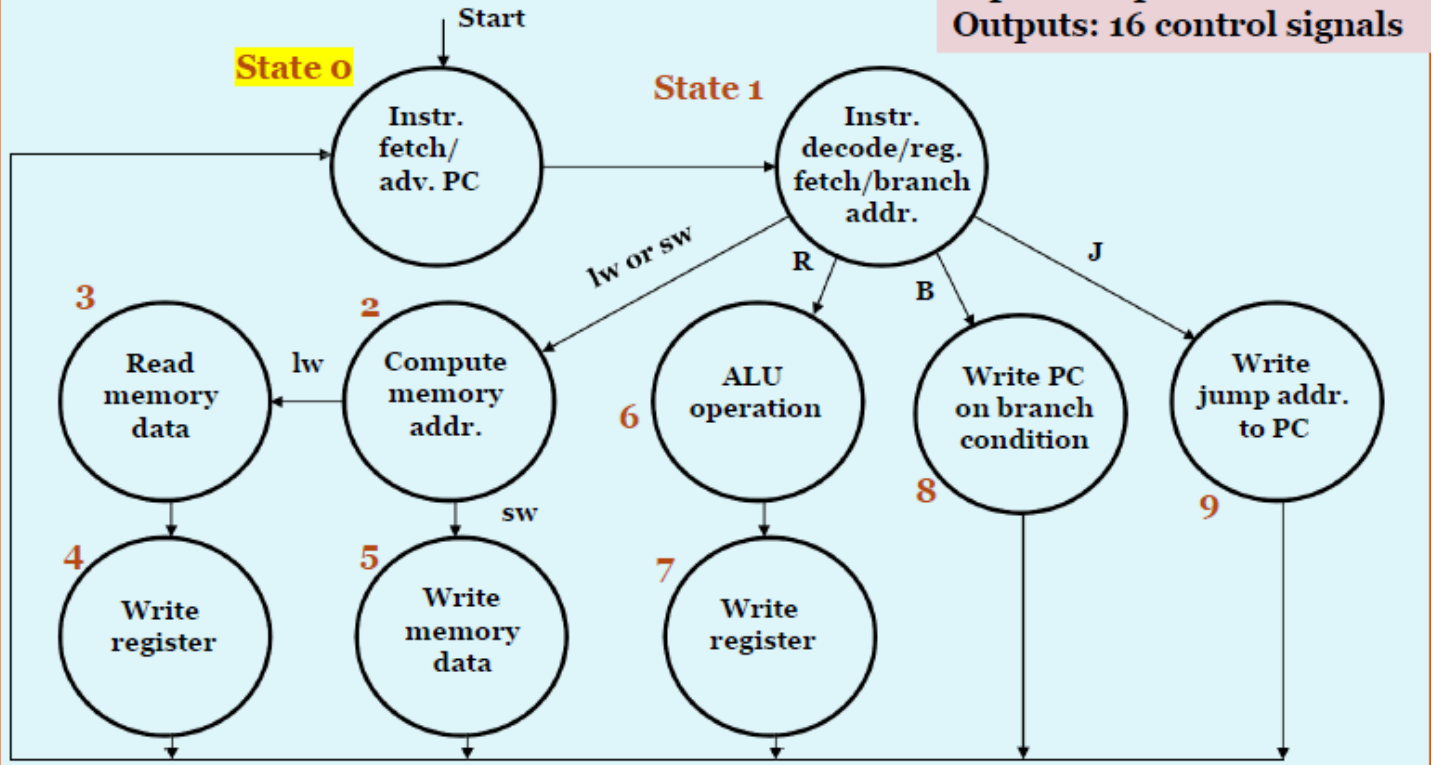
FSM-B: تعليمات الفرع (Branch)

FSM-J: تعليمات القفز (Jump)

Multi-Cycle FSM Control

30

Inputs: 6 opcode bits
Outputs: 16 control signals



■ المدخلات و المخرجات :

Inputs : 6 بت من ال Opcode (رمز التشغيل)

Outputs : 16 إشارة مختلفة

تسلسل الحالات داخل ال FSM:

الحالة	الوظيفة المنفذة في هذه الحالة
0	جلب التعليمات من الذاكرة إلى سجل IR ، وتحديث $PC \leftarrow PC + 4$
1	فك التعليمات، جلب السجلات A و B، وحساب عنوان الفرع إن وجد
2	تنفيذ العملية في ALU أو حساب عنوان الذاكرة
3	تنفيذ الفرع أو القفز أو الوصول إلى الذاكرة
4	كتابة في السجل أو في الذاكرة
5-9	حالات إضافية حسب نوع التعليمات مثل lw ، sw ، R-type ، Branch ، Jump

إشارات التحكم الناتجة :

1. PCWrite : كتابة في عداد البرنامج
2. MemRead : قراءة من الذاكرة
3. MemWrite : كتابة في الذاكرة
4. RegWrite : كتابة في سجل
5. ALUOp : تحديد عملية ALU
6. ALUSrcA, ALUSrcB : تحديد مصادر مدخلات ALU
7. PCSource : تحديد مصدر القيمة الجديدة لـ PC

إشارات التحكم الـ 16:

اسم الإشارة	عدد البتات	الوظيفة (إضافة أربسيزية):
lorD	1	يحدد مصدر عنوان الذاكرة: من PC أو من ALUOut
MemRead	1	تفعيل قراءة من الذاكرة
MemWrite	1	تفعيل كتابة إلى الذاكرة
IRWrite	1	كتابة في سجل التعليمات (Instruction Register)
RegDst	1	اختيار سجل الوجهة rd أو rt
Mem2Reg	1	اختيار مصدر البيانات التي ستكتب في السجل: من ALU أو من الذاكرة
RegWrite	1	تفعيل الكتابة في سجل
ALUSrcA	1	اختيار مدخل A في ALU من PC أو من سجل
ALUSrcB	2	اختيار مدخل B في ALU من سجل، ثابت 4، الإزاحة، أو الإزاحة بعد الإزاحة بـ 2
ALUOp	2	تحديد نوع العملية التي ستنفذها ALU
PCSrc	2	اختيار مصدر القيمة الجديدة لـ PC: من ALU، ALUOut، أو عنوان القفز
PCWrite	1	تفعيل الكتابة في عداد البرنامج (PC)
PCWriteCond	1	تفعيل الكتابة في PC إذا تحقق شرط الفرع مثل (A == B)

تصميم وحدة التحكم (Control Unit) لمعالج MIPS متعدد الدورات (Multi-Cycle)

سنقوم بتصميم وحدة التحكم (Control Unit) لمعالج MIPS متعدد الدورات (Multi-Cycle) وبالتحديد ماذا يحدث في كل دورة (Cycle) من تنفيذ التعليمة .
في الـ Multi-Cycle MIPS التعليمة الواحدة لا تُنفَّذ بدورة واحدة بل تقسّم إلى 5 دورات كما ذكرنا سابقا وسنبداً بها بالتفصيل:

1. الدورة الأولى: جلب التعليمة (Instruction Fetch - IF)

- في هذه المرحلة، يتم جلب التعليمة من الذاكرة ووضعها في سجل التعليمات (IR).
- يتم قراءة التعليمة من العنوان الموجود في عداد البرنامج (PC) وتخزينها في الـ IR باستخدام إشارات التحكم MemRead و IRWrite.
- تحديث الـ PC: يتم زيادة قيمة عداد البرنامج بمقدار 4 ($PC + 4$) ليؤشر على التعليمة التالية في الدورة القادمة

2. الدورة الثانية: تحليل التعليمة (Instruction Decode - ID)

- تقوم وحدة التحكم في هذه المرحلة ب تحليل التعليمة وفك شيفرتها لتحديد نوع العملية المطلوبة.
- يتم قراءة السجلات المطلوبة (Source Registers) وتخزين قيمها في سجلات مؤقتة A و B
- حساب عنوان التفرع: إذا كانت التعليمة من نوع التفرع (Branch)، يتم حساب عنوان التفرع المحتمل في هذه الدورة وتخزينه في سجل ALUOut.

3. الدورة الثالثة: التنفيذ (Execute - EX)

- هذه الدورة هي التي يتم فيها إجراء العمليات الحسابية أو المنطقية، وتختلف وظيفتها حسب نوع التعليمة
- تعليمات R-type: تقوم وحدة الحساب والمنطق (ALU) بتنفيذ العملية على القيم الموجودة في السجلات A و B.
- تعليمات الذاكرة (lw/sw): تقوم الـ ALU بحساب عنوان الذاكرة الفعلي.
- تعليمات التفرع (beq): تقوم الـ ALU بطرح القيمتين للمقارنة، وإذا كانتا متساويتين، يتم تحديث الـ PC بالعنوان الجديد.
- تعليمات القفز (Jump): يتم كتابة عنوان القفز مباشرة في الـ PC.

4. الدورة الرابعة: الوصول للذاكرة أو الكتابة في السجلات (Reg Write/Memory)

- تتعتمد هذه الخطوة على ما تم إنجازه في دورة التنفيذ:
- تعليمات R-type: يتم كتابة نتيجة العملية الحسابية من سجل ALUOut إلى السجل الهدف (Destination Register).
- تعليمية التحميل (lw): يتم قراءة البيانات من الذاكرة (باستخدام العنوان المحسوب) ووضعها في سجل بيانات الذاكرة (MDR).
- تعليمية التخزين (sw): يتم كتابة القيمة الموجودة في سجل B مباشرة إلى الذاكرة.

5. الدورة الخامسة: كتابة السجل (Reg Write)

- هذه الدورة مخصصة فقط لتعليمات التحميل من الذاكرة (lw).
- يتم نقل البيانات التي تم جلبها في الدورة السابقة من سجل الـ MDR وكتابتها في السجل النهائي المحدد في التعليمية.
- بانتهاء هذه الدورة، تعتبر عملية التحميل قد اكتملت تماماً.

ملاحظة:

تعتمد إشارات التحكم (Control Signals) في المعالج متعدد الدورات على نوع المهمة التي يتم تنفيذها في كل مرحلة.

مثلاً :

1. دورة جلب التعليمية (Cycle 1: IF)

تستخدم الإشارات التالية لجلب التعليمية وتحديث عداد البرنامج (PC):

لجلب التعليمية:

- $I \text{ or } D = 0$: لاختيار الـ PC كعنوان للذاكرة.
- $MemRead = 1$: لتفعيل قراءة الذاكرة.
- $IRWrite = 1$: لكتابة التعليمية المجلوبة في سجل التعليمات (IR).



تحديث عداد البرنامج (PC+4):

- $ALUSrcA = 0$: لاختيار الـ PC كمدخل أول للـ ALU.
- $ALUSrcB = 01$: لاختيار الثابت (4) كمدخل ثانٍ للـ ALU.
- $ALUOp = 00$: لتوجيه الـ ALU للقيام بعملية الجمع.
- $PCSource = 00$: لاختيار مخرج الـ ALU ليُعاد إلى الـ PC.
- $PCWrite = 1$: للسماح بتحديث الـ PC.

2. دورة تحليل التعليم (Cycle 2: ID)

في هذه المرحلة، لا تحتاج قراءة السجلات إلى إشارات تحكم، ولكن يتم تجهيز عنوان التفرع المحتمل:

- $ALUSrcA = 0$: اختيار الـ PC.
- $ALUSrcB = 11$: اختيار إزاحة البتات (0-15) للتعليم لاستخدامها في حساب العنوان.
- $ALUOp = 00$: لجمع هذه القيم وحساب عنوان التفرع وتخزينه في $ALUOut$.

3. دورة التنفيذ (Cycle 3: EX)

تختلف الإشارات هنا تماماً حسب نوع التعليم [1، 2]:

- العمليات الحسابية (R-type): $ALUSrcA=1, ALUSrcB=00, ALUOp=10$.
- حساب عنوان الذاكرة (lw/sw): $ALUSrcA=1, ALUSrcB=10, ALUOp=00$.
- التفرع (beq): $ALUSrcA=1, ALUSrcB=00, ALUOp=01, PCSource=01$, و $PCWriteCond = 1$ (ليتم التحديث فقط إذا كان الشرط صحيحاً).
- $PCSource=10, PCWrite = 1$: (Jump)

4. دورة الذاكرة/كتابة السجل (Cycle 4: Memory/Reg Write)

في تعليمات R-type:

- $RegDst = 1$: لاختيار البتات 11-15 كسجل هدف.
- $Mem2Reg = 0$: لاختيار مخرج الـ ALU للكتابة في السجل.
- $RegWrite = 1$: لتنفيذ عملية الكتابة.

في تعليمات الذاكرة:

- التحميل (lw): $I \text{ or } D = 1$ و $MemRead = 1$.
- التخزين (sw): $I \text{ or } D = 1$ و $MemWrite = 1$ لتنفيذ الكتابة في الذاكرة.

5. دورة كتابة السجل (Cycle 5: Reg Write)

هذه الدورة خاصة بتعليمات التحميل (lw) فقط لنقل البيانات من الذاكرة للسجل:

- RegDst = 0: لاختيار البتات 16-20 كسجل هدف.
 - Mem2Reg = 1: لاختيار البيانات القادمة من الذاكرة (MDR) للكتابة.
 - RegWrite = 1: لتنفيذ عملية الكتابة النهائية في السجل.
- تعمل هذه الإشارات (سواء كانت 1-bit أو 2-bit كمفاتيح توجيه للبيانات داخل المسار (Datapath) لضمان تنفيذ كل تعليمة بشكل صحيح.

1-bit Control Signals

(38)

Signal name	Value = 0	Value = 1
RegDst	Write reg. # = bit 16-20	Write reg. # = bit 11-15
RegWrite	No action	Write reg. $\leftarrow$ Write data
ALUSrcA	First ALU Operand $\leftarrow$ PC	First ALU Operand $\leftarrow$ Reg. A
MemRead	No action	Mem.Data Output $\leftarrow$ M[Addr.]
MemWrite	No action	M[Addr.] $\leftarrow$ Mem. Data Input
Mem2Reg	Reg.File Write In $\leftarrow$ ALUOut	Reg.File Write In $\leftarrow$ MDR
I or D	Mem. Addr. $\leftarrow$ PC	Mem. Addr. $\leftarrow$ ALUOut
IRWrite	No action	IR $\leftarrow$ Mem.Data Output
PCWrite	No action	PC is written
PCWriteCond	No action	PC is written if zero (ALU) = 1

zero(ALU) ———— PCWriteCond ———— PCWrite ———— PCWrite etc.

هذا الجدول مخصص للإشارات التي تعمل ك مفاتيح بسيطة ولها حالتان فقط (0 أو 1):

- القيمة 0: تعني إما عدم اتخاذ إجراء (No action) أو اختيار مسار معين (مثل اختيار الـ PC كعنوان للذاكرة في إشارة I or D).
- القيمة 1: تعني تفعيل إجراء معين (مثل RegWrite للكتابة في السجلات، أو MemRead للقراءة من الذاكرة) أو اختيار المسار البديل.

أمثلة من الجدول:

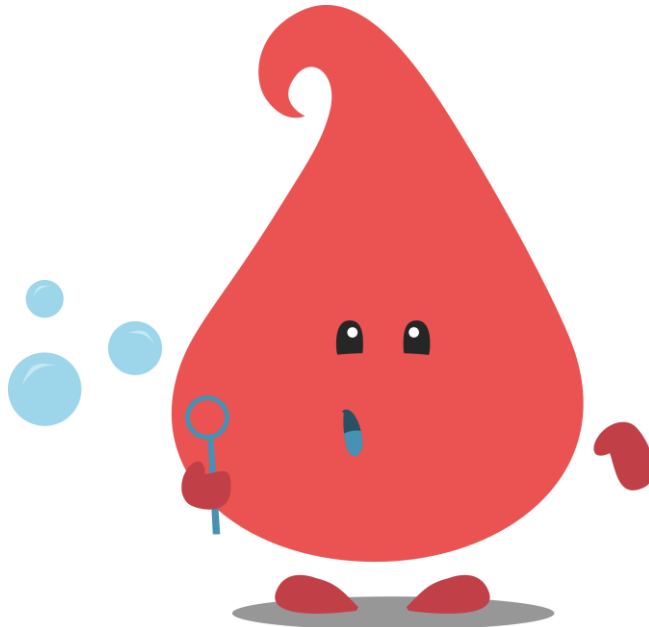
إشارات مثل IRWrite (لحفظ التعليمة) و PCWrite (لتحديث عداد البرنامج) تظهر هنا لأنها ببساطة إما تعمل أو لا تعمل

2-bit Control Signals

Signal name	Value	Action
ALUOp	00	ALU performs add
	01	ALU performs subtract
	10	Funct. field (0-5 bits of IR) determines ALU operation
ALUSrcB	00	Second input of ALU $\leftarrow$ B reg.
	01	Second input of ALU $\leftarrow$ 4 (constant)
	10	Second input of ALU $\leftarrow$ 0-15 bits of IR sign ext. to 32b
	11	Second input of ALU $\leftarrow$ 0-15 bits of IR sign ext. and left shift 2 bits
PCSource	00	ALU output (PC +4) sent to PC
	01	ALUOut (branch target addr.) sent to PC

هذا الجدول مخصص للإشارات الأكثر تعقيداً التي تحتاج للاختيار من بين أكثر من خيارين (تصل إلى 4 خيارات: 00، 01، 10، 11)

- ALUOp: تحدد نوع العملية التي ستقوم بها الـ ALU 00 للجمع، 01 للطرح، 10 للعمليات الحسابية الأخرى
- ALUSrcB: تختار المصدر الثاني للـ ALU من بين 4 خيارات السجل B، أو الثابت 4
- PCSource: تحدد المصدر الذي سيحدث قيمة عداد البرنامج (PC) سواء كان ناتج الـ ALU المباشر، أو عنوان التفرع، أو عنوان القفز.



The end